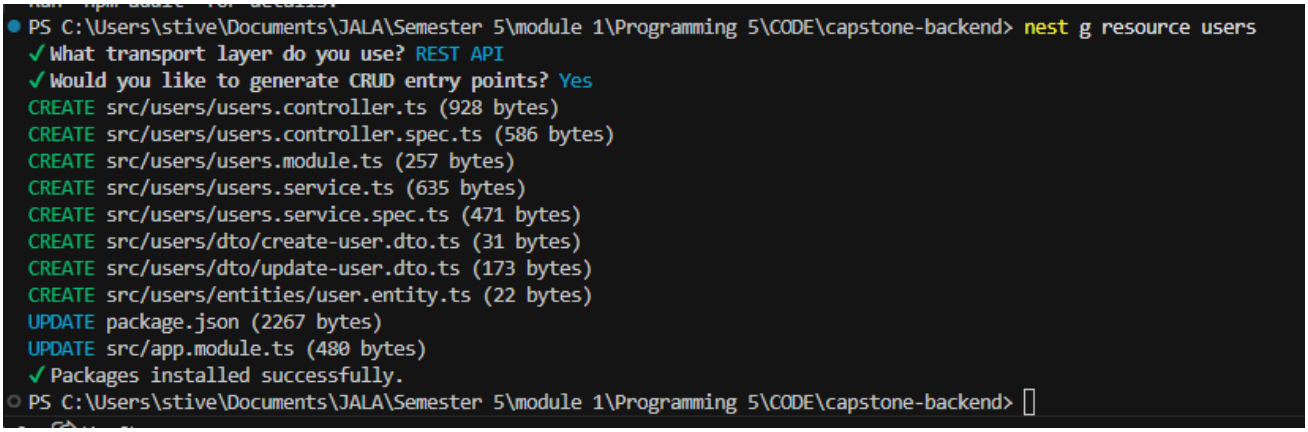
Activity 2

MAde by Cristian Stiven Galeano Barbosa

User Model

- Create the User Schema.



TS create-user.dto.ts U | TS user.entity.ts U

capstone-backend > src > users > entities > TS user.entity.ts > ...

```ts
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';

@Schema()
export class User {
  @Prop({ required: true })
  name: string;

  @Prop({ required: true, unique: true })
  email: string;

  @Prop({ required: true })
  password: string;
}
export const UserSchema = SchemaFactory.createForClass(User);

```

TS create-user.dto.ts U | TS user.entity.ts U

capstone-backend > src > users > dto > TS create-user.dto.ts > CreateUserDto > name

```ts
export class CreateUserDto {
  @IsString()
  readonly name: string;

  @IsString()
  @IsEmail()
  readonly email: string;

  @IsString()
  @IsStrongPassword()
  @IsNotEmpty()
  @MinLength(8)
  readonly password: string;
}

```

- Use bcrypt to hash passwords before saving (pre-save hook).

```
async create(createUserDto: CreateUserDto): Promise<User> {
  try {
    const hashedPassword = await bcrypt.hash(createUserDto.password, 10);
    const createdUser = new this.userModel({
      ...createUserDto,
      password: hashedPassword,
    });
    return await createdUser.save();
  } catch (error) {
    throw new HttpException(`${error}`, HttpStatus.BAD_REQUEST);
  }
}
```

Passport Strategies

- Implement LocalStrategy for login (validate email/password).

```
  async loginUser(email: string, password: string) {
    try {
      const user = await this.userModel.findOne({ email });
      const isPasswordValid = await bcrypt.compare(password, user!.password);

      if (!isPasswordValid) {
        throw new HttpException('Invalid Credentials', HttpStatus.BAD_REQUEST);
      }

      if (user && isPasswordValid) {
        const payload = { email: user.email, sub: user._id, name: user.name };
        return {
          bearer: await this.jwtService.signAsync(payload),
        };
      }
    } catch (error) {
      throw new HttpException(`${error}`, HttpStatus.BAD_REQUEST);
    }
  }
```

- Implement JwtStrategy to protect private routes.

First, we installed jwt to start its configuration.

```
PS C:\Users\stive\Documents\JALA\Semester 5\module 1\Programming 5\CODE\capstone-backend> npm install --save @nestjs/jwt

added 15 packages, and audited 780 packages in 4s

147 packages are looking for funding
  run `npm fund` for details

7 low severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\stive\Documents\JALA\Semester 5\module 1\Programming 5\CODE\capstone-backend>
```

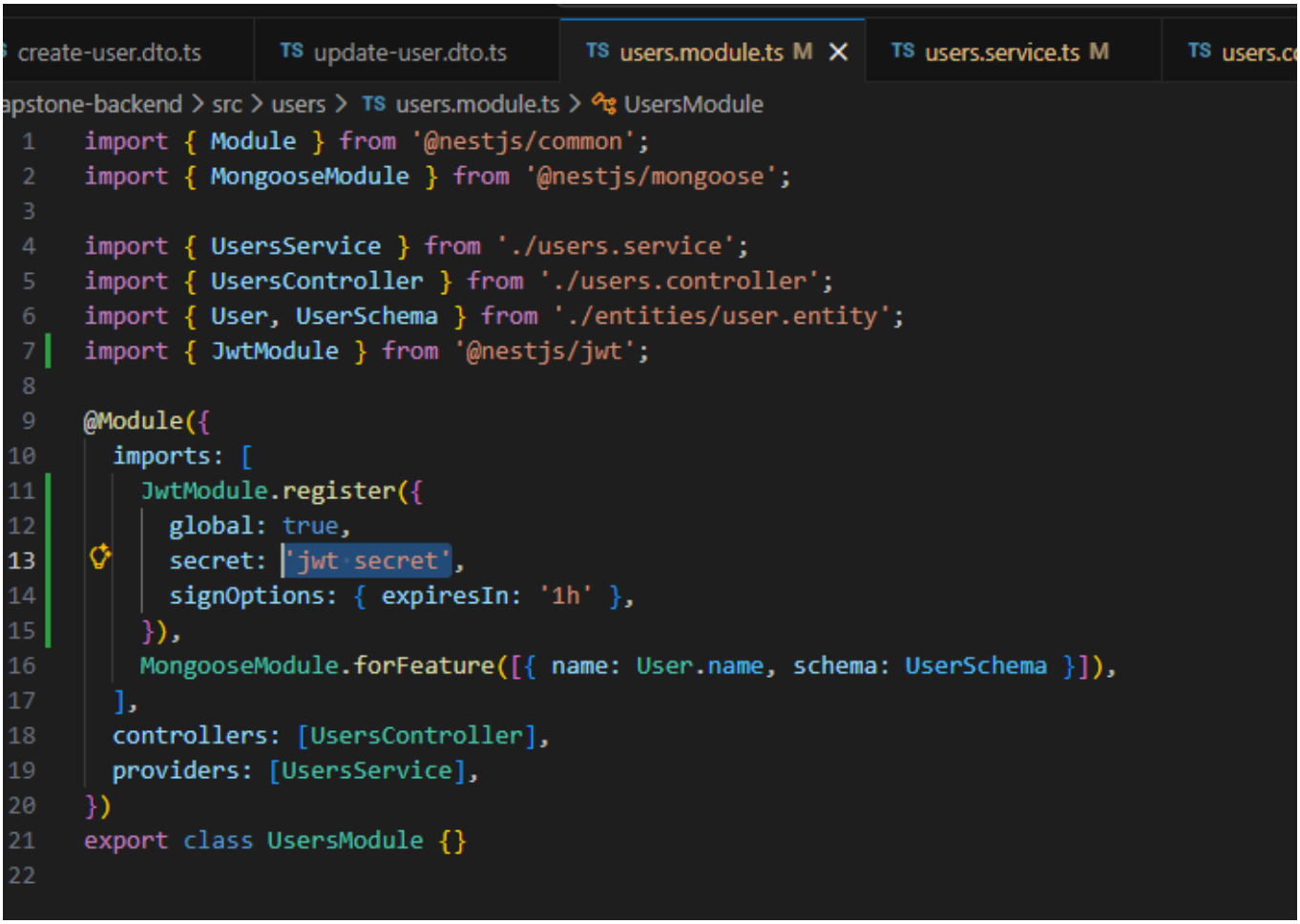
We use it to generate the token when the login is successful.

```
async loginUser(email: string, password: string) {
  try {
    const user = await this.userModel.findOne({ email });
    const isPasswordValid = await bcrypt.compare(password, user!.password);

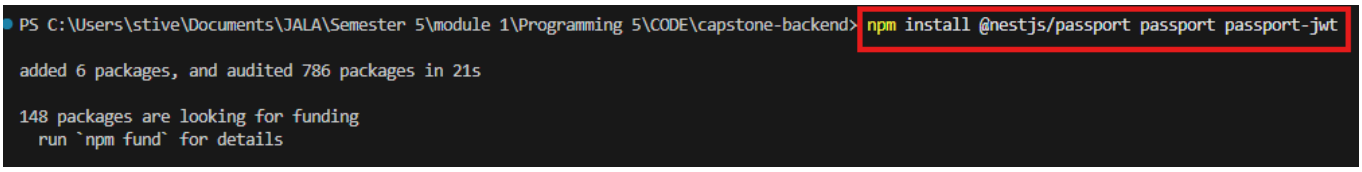
    if (!isPasswordValid) {
      throw new HttpException('Invalid Credentials', HttpStatus.BAD_REQUEST);
    }

    if (user && isPasswordValid) {
      const payload = { email: user.email, sub: user._id, name: user.name };
      return {
        bearer: await this.jwtService.signAsync(payload),
      };
    }
  } catch (error) {
    throw new HttpException(`${error}`, HttpStatus.BAD_REQUEST);
  }
}
```

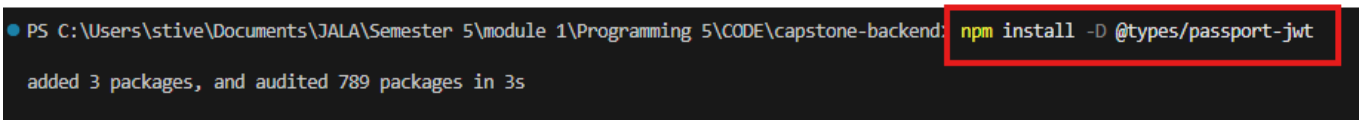
As we saw in class, the token created with the login will last one hour.



To handle private routes, I will use Passport, which will verify that the token is correct to access these private routes.

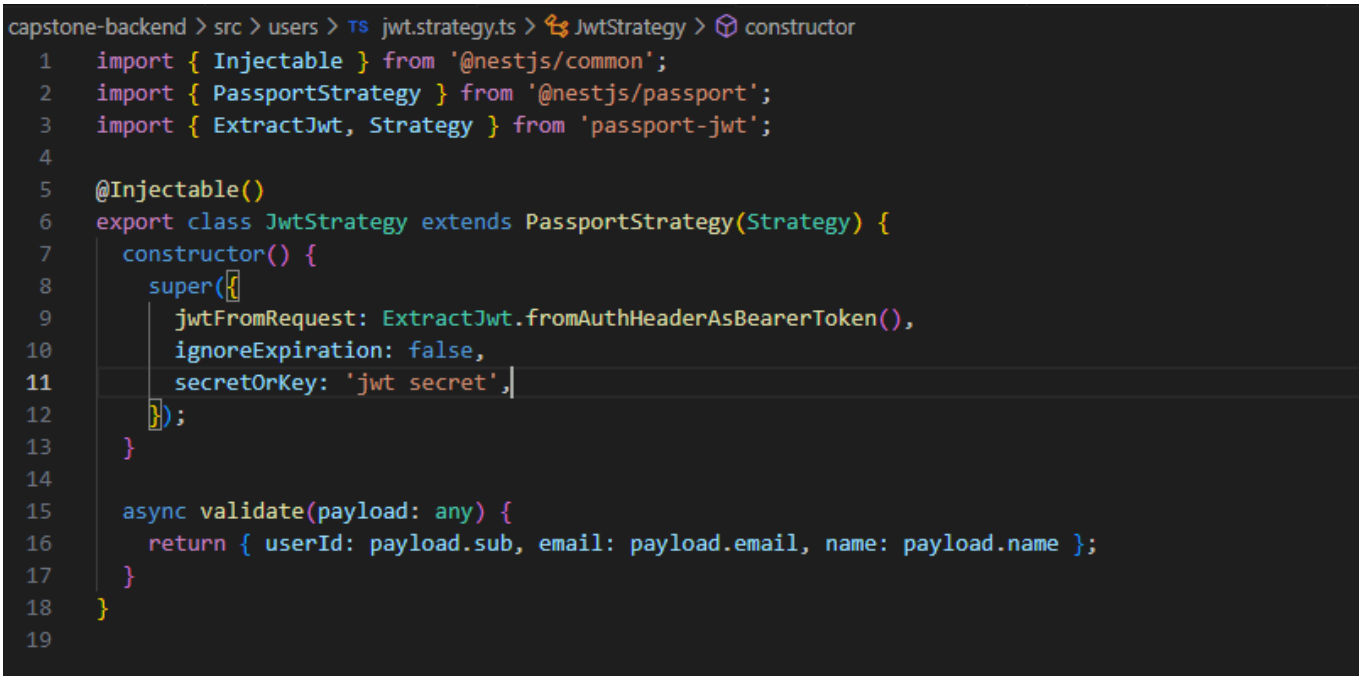


We also installed the Passport types.



We will use JwtStrategy as requested by the lab. This file, based on the bearer token in the

authorization header, searches for the jwt secret key that we used in class for Users.



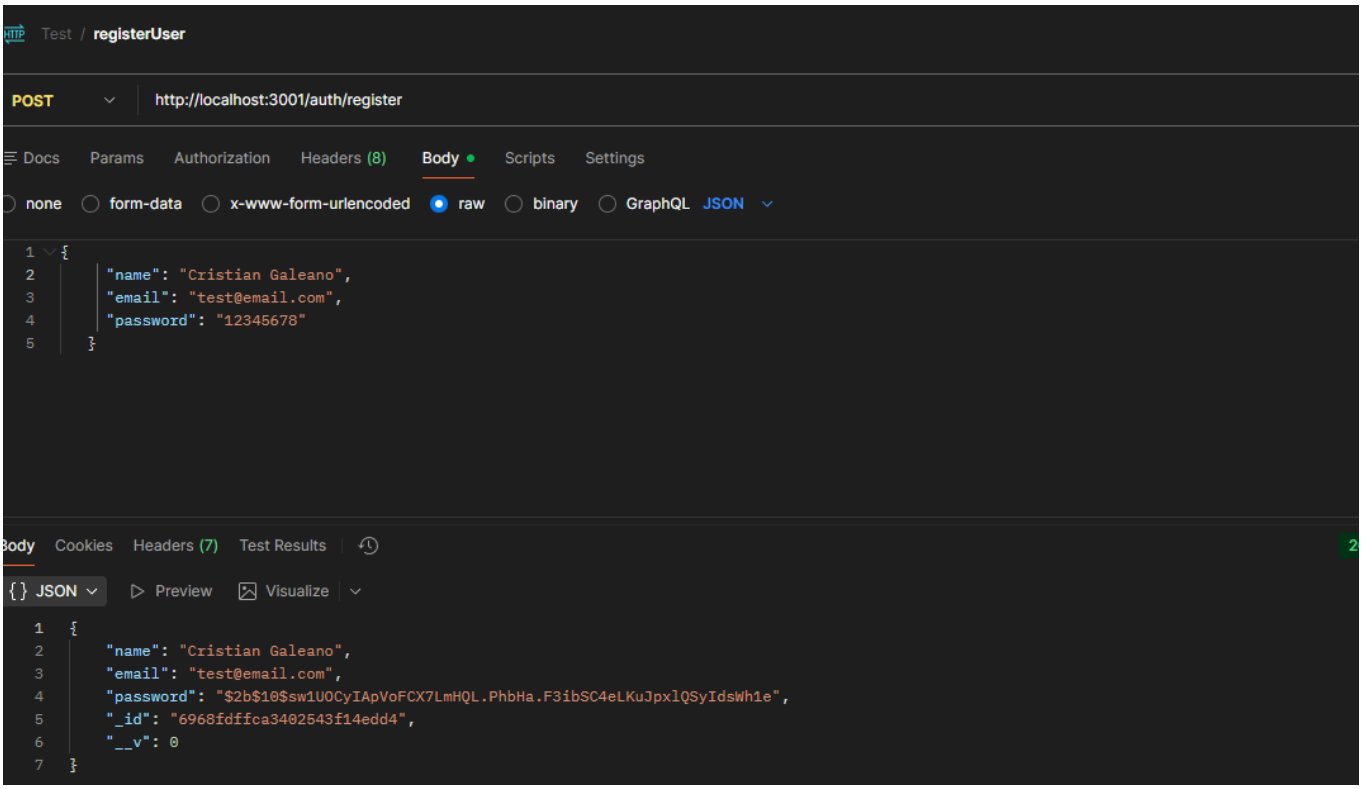
I use this Guard when I want to verify any route in the controller using the strategy created in the previous image.



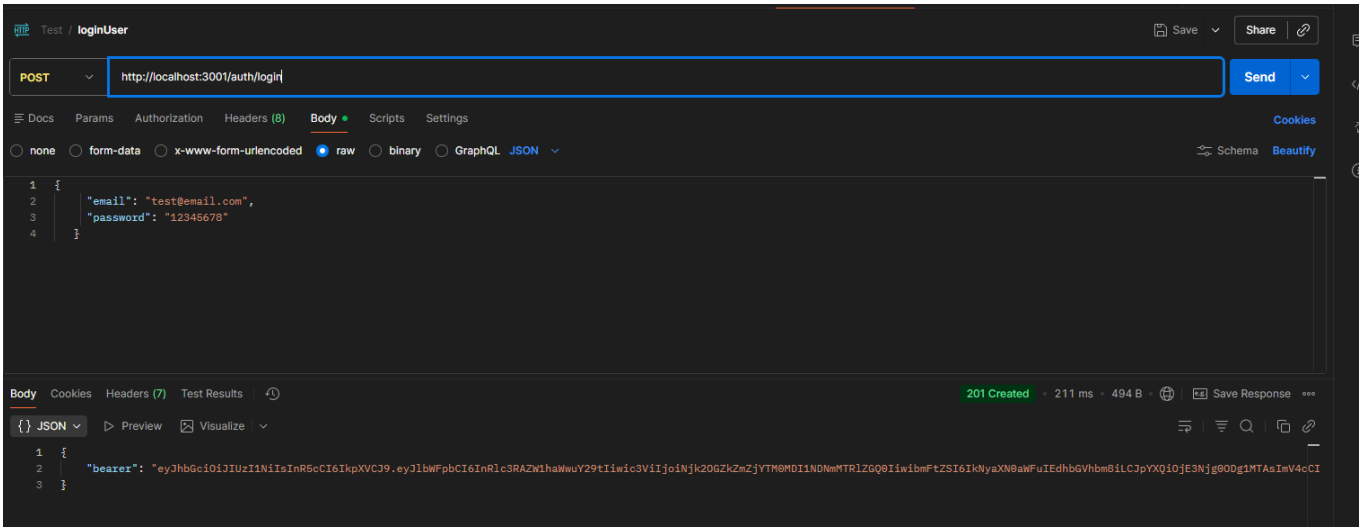
Further on, in the endpoint that must be protected, I explain how to use it.

Controllers and Services

- POST /auth/register

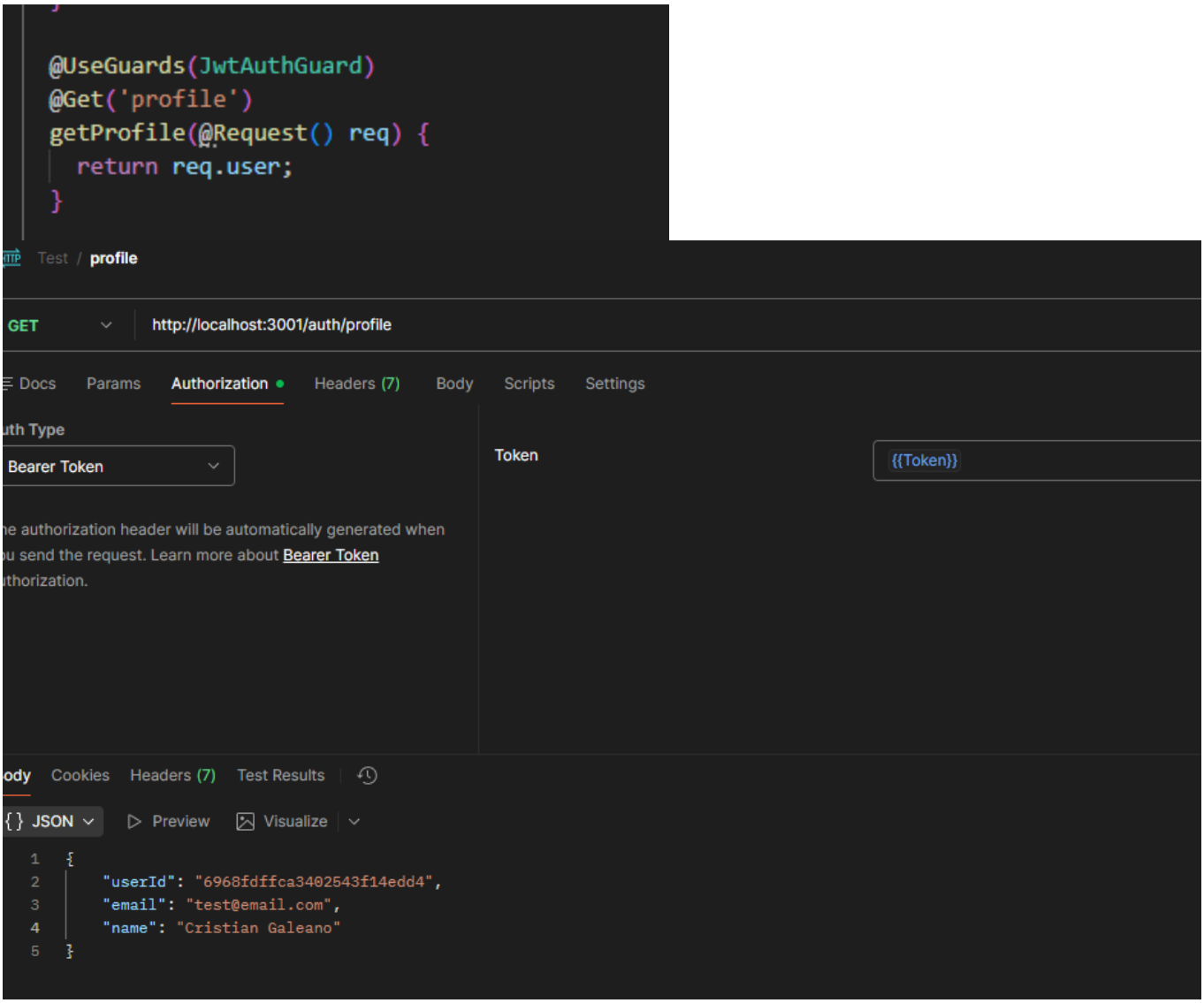


POST /auth/login.



- Protected test endpoint: GET /auth/profile (Only accessible with a valid Token).

In this protected endpoint, we activate it using UseGuards, which essentially calls the doorman and returns all the information injected by jwt.strategy.ts.



Technical Investigation

Why a specific expiration was chosen for the Token

For security, every token must have an expiration, and if it is intended to work all the time, it must be renewed. This is to prevent a token that is no longer in use from continuing to function with the user's permissions within the system, because a man-in-the-middle attack could occur, and this way the risk is limited.

How "Salt" works in bcrypt.

It is a string of random characters that is added to the password before being passed through the hash function. Its main objective is for two identical passwords to generate different hashes.

MR https://gitlab.com/jala-university1/cohort-3/ES.CSPR-351.GA.T1.26.M1/SB/c.galeano/-/merge_requests/2